STM32 Hardware JPEG Peripheral

Image Encoding — Complete Developer Guide

Based on STM32H5 JPEG_EncodingFromFLASH_DMA Example

Covers: STM32F7 · STM32H7 · STM32H5

1. Introduction

The STM32 hardware JPEG codec peripheral is a dedicated hardware accelerator that can encode and decode JPEG images without CPU involvement in the pixel processing. This guide explains every technical aspect of using the peripheral for encoding, based on an analysis of the official STM32H5 DMA-based encoding example.

1.1 What the Hardware JPEG Peripheral Does

JPEG compression involves three computationally expensive steps: the Discrete Cosine Transform (DCT), quantization, and Huffman entropy encoding. On a software-only implementation these steps consume significant CPU time. The JPEG hardware peripheral executes all three steps autonomously in silicon, leaving the CPU free for other tasks.

The peripheral works on blocks of pixels called MCUs (Minimum Coding Units). It accepts raw YCbCr pixel data through a DMA input channel, compresses it in hardware, and emits a JPEG bitstream through a DMA output channel. The CPU role during encoding is limited to buffer management.

1.2 Which STM32 Families Include This Peripheral

The hardware JPEG codec is only available in three STM32 families. Before starting a project, always verify the specific part number in the device datasheet peripheral table.

Family	Lines WITH JPEG	Lines WITHOUT JPEG	Max Clock	DMA Type
STM32F7	F76x, F77x only	All other F7 lines	216 MHz	DMA2 Streams
STM32H7	H743/753, H745/755, H747/757, H750, H7A3/B3/B0, H7Rx/Sx	H723, H725, H730, H733, H735	480 MHz	MDMA / DMA2
STM32H5	H5E5, H5F5 only	H503, H523, H533, H562, H563, H573	250 MHz	GPDMA1

> ⚠ **Note:** All other STM32 families (F4, G4, L4, U5, C0, WB, etc.) have no hardware JPEG peripheral. Software libraries such as libjpeg-turbo must be used instead.

1.3 DMA2D Chrom-Art Advantage on H7 and H5

On STM32F7, the conversion from YCbCr back to RGB after decoding must be performed in software. On STM32H7 and STM32H5 (E5/F5), the DMA2D Chrom-Art Accelerator can perform this conversion in hardware, significantly accelerating decode-to-display pipelines. The STM32H5 features an enhanced Chrom-ART2 version.

2. JPEG Fundamentals for Embedded Developers

2.1 Why YCbCr Instead of RGB

JPEG does not work in RGB color space. Before encoding, pixel data must be converted to YCbCr. This is a critical step performed by the STM32 JPEG utility library before the data is handed to the hardware:

- Y — Luminance (brightness). The human eye is very sensitive to brightness changes.
- Cb — Chrominance blue: how blue the pixel is relative to neutral grey.
- Cr — Chrominance red: how red the pixel is relative to neutral grey.

The key insight enabling JPEG compression is that the human eye is far less sensitive to color variations than to brightness variations. Chroma channels (Cb, Cr) can therefore be reduced with minimal perceptible quality loss — this is called chroma subsampling.

2.2 Chroma Subsampling: 4:4:4, 4:2:2, and 4:2:0

The notation A:B:C describes how many chroma samples are kept relative to luma samples per row. In all three modes every luma (Y) sample is preserved. Only the chroma density changes.

Mode	Description	Cb/Cr Coverage	MCU Block Size	MAX_INPUT_LINES	Relative File Size
4:4:4	No subsampling	1 sample per pixel	8 × 8 px	8	Largest
4:2:2	Halved horizontally	1 sample per 2 pixels (horizontal)	16 × 8 px	8	Medium (~75% of 4:4:4)
4:2:0	Halved both axes	1 sample per 2×2 pixel block	16 × 16 px	16	Smallest (~50% of 4:4:4)

In 4:2:0 mode, a single Cb and a single Cr value is shared by a 2×2 block of four pixels. One chroma sample therefore covers two rows of luma, which is why the MCU block height doubles to 16 lines.

2.3 The MCU (Minimum Coding Unit)

The JPEG hardware always processes data in MCU blocks. An MCU contains all the data needed to encode one rectangular tile of the image. The hardware cannot encode a partial MCU — input data must always be provided as complete MCU rows.

Mode	MCU Content	MCU Pixel Size	Y blocks	Cb blocks	Cr blocks
4:4:4	3 independent 8×8 blocks	8 × 8 px	1	1	1
4:2:2	2 Y blocks + 1 Cb + 1 Cr	16 × 8 px	2 (side by side)	1	1
4:2:0	4 Y blocks + 1 Cb + 1 Cr	16 × 16 px	4 (2×2 arrangement)	1	1

2.4 Why MAX_INPUT_LINES Must Be 16 for 4:2:0 and 8 for Others

The input to the JPEG hardware must contain exactly one complete row of MCUs in the height dimension. Since an MCU tile is 16 pixels tall in 4:2:0 (because the single chroma sample spans two pixel rows), the input chunk must be exactly 16 lines. In 4:4:4 and 4:2:2 the MCU is only 8 pixels tall, so 8 lines suffice.

```
// main.h — this value drives buffer sizes and the DMA chunk loop
#define MAX_INPUT_LINES  16   // 16 for YCbCr 4:2:0
                              // 8  for YCbCr 4:2:2 and 4:4:4

// DataBufferSize per chunk — how many raw RGB bytes to read from FLASH
DataBufferSize = Conf.ImageWidth * MAX_INPUT_LINES * BYTES_PER_PIXEL;
// Example (320x240, ARGB8888, 4:2:0): 320 x 16 x 4 = 20 480 bytes
```

> ⚠ **Note:** Setting MAX_INPUT_LINES = 8 with 4:2:0 subsampling will cause the hardware to receive half an MCU block and produce corrupted JPEG output. The RGB_GetInfo() validation function in the example code calls Error_Handler() if image dimensions are not multiples of 8 (always) and multiples of 16 where required.

3. Configuration Parameters

3.1 jpeg_utils_conf.h — Color Format

This header selects the pixel format of the source image. It drives the BYTES_PER_PIXEL macro and the selection of the correct RGB-to-YCbCr conversion function.

```
// jpeg_utils_conf.h
#define JPEG_ARGB8888   0   // 4 bytes per pixel: Alpha, Red, Green, Blue
#define JPEG_RGB888     1   // 3 bytes per pixel: Red, Green, Blue
#define JPEG_RGB565     2   // 2 bytes per pixel: 5-bit R, 6-bit G, 5-bit B

#define JPEG_RGB_FORMAT  JPEG_ARGB8888  // ← change this for your image format
#define JPEG_SWAP_RG     0              // Set to 1 for BGR-order cameras
#define USE_JPEG_ENCODER 1
#define USE_JPEG_DECODER 1
```

Format	BYTES_PER_PIXEL	Chunk size (320×16)	Common source
JPEG_ARGB8888	4	20 480 bytes	LCD framebuffer, DMA2D output
JPEG_RGB888	3	15 360 bytes	Camera (OV7670 RGB mode)
JPEG_RGB565	2	10 240 bytes	LCD framebuffer, camera (OV7670 RGB565)

3.2 main.h — Encoding Parameters

```
// main.h — all tuneable encoding parameters
#define JPEG_CHROMA_SAMPLING  JPEG_420_SUBSAMPLING   // 4:2:0 — most compressed
                                                     // JPEG_422_SUBSAMPLING
                                                     // JPEG_444_SUBSAMPLING

#define JPEG_COLOR_SPACE      JPEG_YCBCR_COLORSPACE  // standard color JPEG
                                                     //
JPEG_GRAYSCALE_COLORSPACE
                                                     // JPEG_CMYK_COLORSPACE

#define JPEG_IMAGE_QUALITY    75   // 0 = worst / smallest, 100 = lossless /
largest

#define MAX_INPUT_WIDTH       800  // maximum image width this build supports
#define MAX_INPUT_LINES       16   // 16 for 4:2:0 — 8 for 4:2:2 and 4:4:4
```

3.3 Image Size Validation

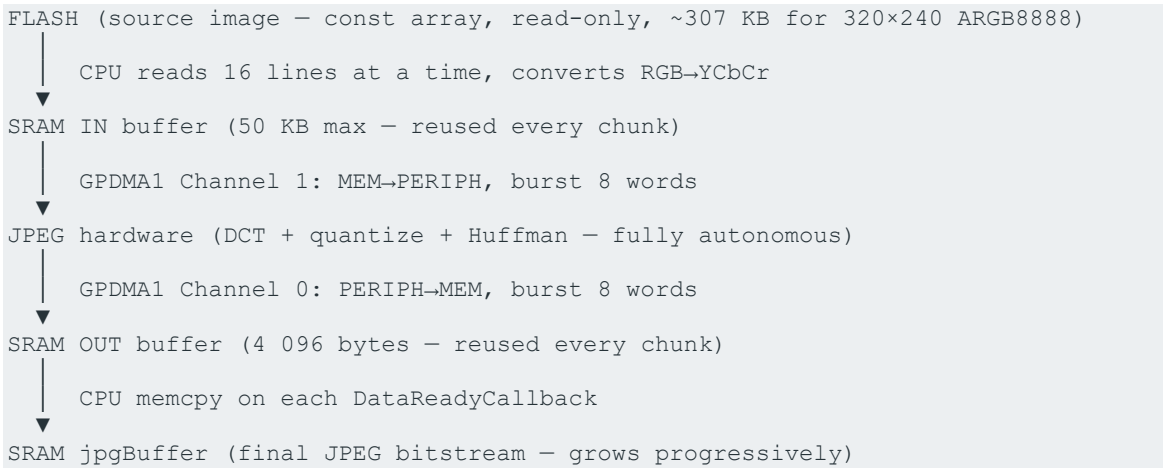
The RGB_GetInfo() function validates image dimensions before encoding starts. It calls Error_Handler() if any of these four rules is violated:

Rule	Condition	Reason
Width multiple of 8	Always required	DCT operates on 8×8 blocks — partial blocks are invalid
Height multiple of 8	Always required	Same reason as above
Width multiple of 16	YCbCr 4:2:2 and 4:2:0	MCU width is 16 pixels — half-MCUs cannot be encoded
Height multiple of 16	YCbCr 4:2:0 only	MCU height is 16 pixels in 4:2:0 mode

4. System Architecture

4.1 Overview

The encoding pipeline is built around three layers working concurrently: the CPU running a background state machine, two GPDMA1 channels handling data movement, and the JPEG hardware performing compression. No layer blocks the others.



4.2 Memory Budget

Buffer	Size formula	Example (320×240 ARGB8888 4:2:0)	Purpose
MCU_Data_IntBuffer0	MAX_INPUT_WIDTH × MAX_INPUT_LINES × BPP	51 200 bytes	Input chunk for DMA Channel 1
MCU_Data_InBuffer1	Same as above	51 200 bytes	Declared but unused — single-buffer scheme
JPEG_Data_OutBuffer0	CHUNK_SIZE_OUT = 4 096	4 096 bytes	Output chunk for DMA Channel 0
JPEG_Data_OutBuffer1	Same as above	4 096 bytes	Declared but unused
jpgBuffer[]	RGB_IMAGE_SIZE (worst case)	307 200 bytes	Final JPEG bitstream accumulator
Total SRAM used		~413 KB	Verify fits in target MCU SRAM

> ⚠ **Note:** jpgBuffer is sized to the raw input size as a worst-case guarantee. Real JPEG output is typically 5-20x smaller. If RAM is tight, reduce jpgBuffer to image_size/4 and add a bounds check.

4.3 Chunk Processing Loop

For a 320×240 image in 4:2:0 mode the complete processing is:

- Image height: 240 lines, MCU height: 16 lines → 15 chunks (iterations)
- Image width: 320 pixels, MCU width: 16 pixels → 20 MCUs per chunk row
- Total MCUs: 20 × 15 = 300 MCUs (this is MCU_TotalNb)
- Each chunk: 20 480 bytes RGB in → variable bytes JPEG out (typically 500–2 000 bytes)
- Each chunk: 20 480 ÷ 32 = 640 DMA bursts → 640 GetDataCallback calls
- Full image: 640 × 15 = 9 600 GetDataCallback calls total

5. Clock and Peripheral Initialization

5.1 System Clock (SystemClock_Config)

The STM32H5F5 is configured at its maximum 250 MHz using an external crystal (HSE). The PLL settings must match your board's crystal frequency.

```
// SystemClock_Config() — STM32H5F5, 48 MHz HSE crystal → 250 MHz SYSCLK
RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
RCC_OscInitStruct.HSEState       = RCC_HSE_ON;
RCC_OscInitStruct.PLL.PLLState   = RCC_PLL_ON;
RCC_OscInitStruct.PLL.PLLSource  = RCC_PLL1_SOURCE_HSE;
RCC_OscInitStruct.PLL.PLLM       = 24;    // 48 MHz / 24 = 2 MHz VCI
RCC_OscInitStruct.PLL.PLLN       = 250;   // 2 MHz × 250 = 500 MHz VCO
RCC_OscInitStruct.PLL.PLLP       = 2;     // 500 MHz / 2 = 250 MHz SYSCLK

// Formula: SYSCLK = (HSE / PLLM) × PLLN / PLLP
// Required: FLASH_LATENCY_5 at 250 MHz + SCALE0 voltage
__HAL_FLASH_SET_PROGRAM_DELAY(FLASH_PROGRAMMING_DELAY_2);
```

Parameter	Value	Notes
Voltage scale	PWR_REGULATOR_VOLTAGE_SCALE0	Highest performance mode — required for 250 MHz
Flash latency	FLASH_LATENCY_5	Required at 250 MHz, Vcore Scale 0 — check datasheet for other speeds
ICACHE	Enabled	Critical for FLASH performance at 250 MHz

> ✓ **Tip:** When porting to a different board, recalculate PLLM/PLLN/PLLP for your HSE crystal frequency. Formula: SYSCLK = (HSE / PLLM) × PLLN / PLLP. Always verify Flash latency in the reference manual for your target frequency.

5.2 JPEG Peripheral Init

```
// MX_JPEG_Init() — simple, all config is in HAL_JPEG_ConfigEncoding later
hjpeg.Instance = JPEG;
HAL_JPEG_Init(&hjpeg);  // triggers HAL_JPEG_MspInit callback automatically
```

5.3 DMA Configuration (HAL_JPEG_MspInit)

This MSP callback is invoked automatically by HAL_JPEG_Init(). It configures the two DMA channels that serve the JPEG peripheral. The configuration is asymmetric because one channel feeds data in and the other drains data out.

```
// Channel 1 — Input: Memory → JPEG peripheral (raw YCbCr MCU data)
```

```
handle_GPDMA1_Channel1.Init.Request             = GPDMA1_REQUEST_JPEG_RX_REQ;
handle_GPDMA1_Channel1.Init.Direction           = DMA_MEMORY_TO_PERIPH;
handle_GPDMA1_Channel1.Init.SrcInc              = DMA_SINC_INCREMENTED;  //
walk through buffer
handle_GPDMA1_Channel1.Init.DestInc             = DMA_DINC_FIXED;        //
always → JPEG FIFO
handle_GPDMA1_Channel1.Init.SrcDataWidth        = DMA_SRC_DATAWIDTH_WORD;
handle_GPDMA1_Channel1.Init.DestDataWidth       = DMA_DEST_DATAWIDTH_WORD;
handle_GPDMA1_Channel1.Init.SrcBurstLength      = 8;  // 8 words = 32 bytes per
burst
handle_GPDMA1_Channel1.Init.DestBurstLength     = 8;
handle_GPDMA1_Channel1.Init.TransferEventMode   = DMA_TCEM_BLOCK_TRANSFER; //
IRQ per burst
handle_GPDMA1_Channel1.Init.TransferAllocatedPort = DMA_SRC_ALLOCATED_PORT0 |
DMA_DEST_ALLOCATED_PORT1;
HAL_DMA_Init(&handle_GPDMA1_Channel1);
__HAL_LINKDMA(hjpeg, hdmain, handle_GPDMA1_Channel1);

// Channel 0 — Output: JPEG peripheral → Memory (compressed JPEG bitstream)
handle_GPDMA1_Channel0.Init.Request             = GPDMA1_REQUEST_JPEG_TX_REQ;
handle_GPDMA1_Channel0.Init.Direction           = DMA_PERIPH_TO_MEMORY;
handle_GPDMA1_Channel0.Init.SrcInc              = DMA_SINC_FIXED;        //
always from JPEG FIFO
handle_GPDMA1_Channel0.Init.DestInc             = DMA_DINC_INCREMENTED;  //
walk through buffer
HAL_DMA_Init(&handle_GPDMA1_Channel0);
__HAL_LINKDMA(hjpeg, hdmaout, handle_GPDMA1_Channel0);
```

The TransferEventMode = DMA_TCEM_BLOCK_TRANSFER is critical. It causes the DMA to fire an IRQ after every burst of 8 words (32 bytes), which triggers HAL_JPEG_GetDataCallback. This allows the JPEG hardware to pace the DMA — the hardware issues JPEG_RX_REQ when ready for more data. Without this mode, the DMA would try to send all data continuously, overflowing the JPEG FIFO.

DMA Ch	Request	Direction	IRQ event	Linked to
Channel 1	JPEG_RX_REQ	MEM → PERIPH	Every 32-byte burst (BLOCK_TRANSFER)	hjpeg.hdmain
Channel 0	JPEG_TX_REQ	PERIPH → MEM	Buffer full (4 096 bytes)	hjpeg.hdmaout

6. The Encoding Function: JPEG_Encode_DMA

6.1 Function Signature and Purpose

```
uint32_t JPEG_Encode_DMA(
    JPEG_HandleTypeDef *hjpeg,
    uint32_t RGBImageBufferAddress,  // address of source image in FLASH
    uint32_t RGBImageSize_Bytes,     // total size of source image in bytes
    uint32_t *jpgBufferAddress       // destination for JPEG output in SRAM
);
```

This function sets up the complete encoding pipeline and fires the first DMA transfer. It returns immediately — encoding continues asynchronously in the background loop.

6.2 Step-by-Step Walkthrough

Step 1 — Save destination pointer

```
pJpegBuffer = jpgBufferAddress;  // global pointer, advances as chunks are
written
```

The destination pointer is saved globally so it can be advanced inside OutputHandler as compressed chunks arrive over time.

Step 2 — Reset all state

```
MCU_TotalNb = 0; MCU_BlockIndex = 0; Jpeg_HWEncodingEnd = 0;
Output_Is_Paused = 0; Input_Is_Paused = 0;
```

All state is reset before each encode call. Essential if encoding multiple images sequentially.

Step 3 — Get image info and select conversion function

```
RGB_GetInfo(&Conf);  // fills width, height, quality, colorspace, subsampling
JPEG_GetEncodeColorConvertFunc(&Conf, &pRGBToYCbCr_Convert_Function,
&MCU_TotalNb);
// pRGBToYCbCr_Convert_Function: function pointer to the correct RGB→YCbCr
converter
// MCU_TotalNb: total MCU block count for this image (e.g. 300 for 320×240 in
4:2:0)
```

JPEG_GetEncodeColorConvertFunc selects from the jpeg_utils library the correct conversion function based on pixel format (ARGB8888/RGB888/RGB565) and subsampling mode. It also computes MCU_TotalNb — the total number of MCU blocks in the full image, used as the end condition.

Step 4 — Pre-process first chunk

```
DataBufferSize = Conf.ImageWidth * MAX_INPUT_LINES * BYTES_PER_PIXEL;
// = 320 × 16 × 4 = 20 480 bytes (source RGB data per chunk)

MCU_BlockIndex += pRGBToYCbCr_Convert_Function(
```

```
    (uint8_t *)(RGB_InputImageAddress + RGB_InputImageIndex),  // source: start
of image
    Jpeg_IN_BufferTab.DataBuffer,                              // destination:
IN buffer
    0,                                                         // start line
offset
    DataBufferSize,                                            // how many
source bytes to process
    (uint32_t*)(&Jpeg_IN_BufferTab.DataBufferSize)             // output: YCbCr
bytes produced
);
Jpeg_IN_BufferTab.State = JPEG_BUFFER_FULL;
RGB_InputImageIndex += DataBufferSize;
```

The first chunk must be pre-processed before HAL_JPEG_Encode_DMA is called, because the DMA requires a filled input buffer to start. This converts the first 16 lines from RGB to YCbCr MCU-ordered data and stores the result size in Jpeg_IN_BufferTab.DataBufferSize.

Note the two different DataBufferSize values:

- DataBufferSize (local): size of RGB source data consumed from FLASH — fixed, always ImageWidth × LINES × BPP
- Jpeg_IN_BufferTab.DataBufferSize (struct field): size of YCbCr output written to IN buffer — variable, computed by the conversion function, used by DMA Channel 1

Step 5 — Configure and start encoding

```
HAL_JPEG_ConfigEncoding(hjpeg, &Conf);
// Writes quality factor, subsampling mode, image dimensions to JPEG HW
registers
// Selects quantization tables based on quality setting

HAL_JPEG_Encode_DMA(
    hjpeg,
    Jpeg_IN_BufferTab.DataBuffer,      // DMA Ch1 source address
    Jpeg_IN_BufferTab.DataBufferSize,  // DMA Ch1 initial transfer size (YCbCr
bytes)
    Jpeg_OUT_BufferTab.DataBuffer,     // DMA Ch0 destination address
    CHUNK_SIZE_OUT                     // DMA Ch0 buffer size = 4 096 bytes
);
return 0;  // returns immediately — encoding continues in background
```

7. The Background Loop and Interrupt Callbacks

7.1 The do/while Loop in main()

```
tickstart = HAL_GetTick();
do {
    JPEG_EncodeInputHandler(&hjpeg);   // prepare next RGB→YCbCr chunk
    jpeg_encode_processing_end = JPEG_EncodeOutputHandler(&hjpeg);  // drain
compressed output
} while ((jpeg_encode_processing_end == 0) && ((HAL_GetTick() - tickstart) <
5000));
```

The CPU loops continuously while the hardware encodes. The two handlers do nothing if their conditions are not met — they simply return on each iteration until a flag changes. The 5-second timeout is a safety net: if the DMA or hardware hangs, the loop exits and the red LED is turned on.

7.2 HAL_JPEG_GetDataCallback — Input DMA Event

This callback is triggered by the DMA Channel 1 IRQ after every burst of 32 bytes (8 words). It is called approximately 640 times per chunk and 9 600 times for a full 320×240 image in 4:2:0.

```
void HAL_JPEG_GetDataCallback(JPEG_HandleTypeDef *hjpeg, uint32_t
NbEncodedData)
{
    if (NbEncodedData == Jpeg_IN_BufferTab.DataBufferSize)
    {
        // All bytes in the current chunk have been consumed by JPEG hardware
        Jpeg_IN_BufferTab.State = JPEG_BUFFER_EMPTY;
        Jpeg_IN_BufferTab.DataBufferSize = 0;
        HAL_JPEG_Pause(hjpeg, JPEG_PAUSE_RESUME_INPUT);  // stop DMA Ch1
        Input_Is_Paused = 1;                              // signal to
InputHandler
    }
    else
    {
        // Only a partial burst consumed — slide the DMA window forward
        HAL_JPEG_ConfigInputBuffer(hjpeg,
            Jpeg_IN_BufferTab.DataBuffer + NbEncodedData,
            Jpeg_IN_BufferTab.DataBufferSize - NbEncodedData);
    }
}
```

The else branch is the common path. It reconfigures the DMA source address and remaining size so the next burst continues from where the last one ended. This is necessary because DMA_TCEM_BLOCK_TRANSFER stops the DMA after each burst and requires explicit re-arming via HAL_JPEG_ConfigInputBuffer.

NbEncodedData value	Meaning	Action taken
< DataBufferSize (bursts 1–639)	Partial: more data remains in buffer	Reconfigure DMA: ptr+N, size-N

== DataBufferSize (burst 640)	Complete: full chunk consumed	Pause DMA Ch1, set EMPTY flag, set Input_Is_Paused=1

7.3 JPEG_EncodeInputHandler — CPU Response to Input Event

```
void JPEG_EncodeInputHandler(JPEG_HandleTypeDef *hjpeg)
{
    uint32_t DataBufferSize = Conf.ImageWidth * MAX_INPUT_LINES *
BYTES_PER_PIXEL;

    // Only act if the IN buffer is empty AND more MCUs need encoding
    if ((Jpeg_IN_BufferTab.State == JPEG_BUFFER_EMPTY) && (MCU_BlockIndex <=
MCU_TotalNb))
    {
        if (RGB_InputImageIndex < RGB_InputImageSize_Bytes)
        {
            // Convert next 16-line chunk from FLASH RGB to YCbCr MCU order
            MCU_BlockIndex += pRGBToYCbCr_Convert_Function(...);
            Jpeg_IN_BufferTab.State = JPEG_BUFFER_FULL;
            RGB_InputImageIndex += DataBufferSize;

            if (Input_Is_Paused == 1)
            {
                Input_Is_Paused = 0;
                // Give DMA Ch1 the new buffer address and size, then restart
                HAL_JPEG_ConfigInputBuffer(hjpeg,
                    Jpeg_IN_BufferTab.DataBuffer,
                    Jpeg_IN_BufferTab.DataBufferSize);
                HAL_JPEG_Resume(hjpeg, JPEG_PAUSE_RESUME_INPUT);
            }
        }
        else
        {
            MCU_BlockIndex++;  // image fully read — increment to unblock exit
condition
        }
    }
}
```

7.4 HAL_JPEG_DataReadyCallback — Output DMA Event

This callback is triggered when the OUT buffer is full — the DMA Channel 0 has written CHUNK_SIZE_OUT (4 096) bytes of compressed JPEG data.

```
void HAL_JPEG_DataReadyCallback(JPEG_HandleTypeDef *hjpeg,
                                uint8_t *pDataOut, uint32_t OutDataLength)
{
    Jpeg_OUT_BufferTab.State = JPEG_BUFFER_FULL;
    Jpeg_OUT_BufferTab.DataBufferSize = OutDataLength;  // actual bytes in this
chunk

    HAL_JPEG_Pause(hjpeg, JPEG_PAUSE_RESUME_OUTPUT);   // stop DMA Ch0
    Output_Is_Paused = 1;

    // Reconfigure output buffer for next chunk (does not restart yet)
```

```
    HAL_JPEG_ConfigOutputBuffer(hjpeg, Jpeg_OUT_BufferTab.DataBuffer,
CHUNK_SIZE_OUT);
}
```

7.5 JPEG_EncodeOutputHandler — CPU Response to Output Event

```
uint32_t JPEG_EncodeOutputHandler(JPEG_HandleTypeDef *hjpeg)
{
    if (Jpeg_OUT_BufferTab.State == JPEG_BUFFER_FULL)
    {
        // Copy compressed chunk to final JPEG buffer
        memcpy(pJpegBuffer, Jpeg_OUT_BufferTab.DataBuffer,
Jpeg_OUT_BufferTab.DataBufferSize);
        pJpegBuffer += Jpeg_OUT_BufferTab.DataBufferSize / 4;  // uint32_t*
advances by words
        Jpeg_OUT_BufferTab.State = JPEG_BUFFER_EMPTY;
        Jpeg_OUT_BufferTab.DataBufferSize = 0;

        if (Jpeg_HWEncodingEnd != 0)
            return 1;  // encoding complete — exit main loop

        else if (Output_Is_Paused == 1 && Jpeg_OUT_BufferTab.State ==
JPEG_BUFFER_EMPTY)
        {
            Output_Is_Paused = 0;
            HAL_JPEG_Resume(hjpeg, JPEG_PAUSE_RESUME_OUTPUT);  // restart DMA
Ch0
        }
    }
    return 0;
}
```

> ⚠ **Note:** HWEncodingEnd is tested AFTER the memcpy, not before. The final chunk of compressed data must be copied before signalling completion. Testing before the memcpy would silently discard the last chunk.

7.6 HAL_JPEG_EncodeCpltCallback

```
void HAL_JPEG_EncodeCpltCallback(JPEG_HandleTypeDef *hjpeg)
{
    Jpeg_HWEncodingEnd = 1;  // detected by OutputHandler on next iteration
}
```

Called from the JPEG peripheral IRQ when hardware encoding is complete. Sets a flag — all actual work (the final memcpy) is done by OutputHandler in the CPU context on the next loop iteration.

8. Interrupt and IRQ Architecture

8.1 IRQ Handlers Required

```
// stm32h5xx_it.c — three IRQ handlers are required

void GPDMA1_Channel1_IRQHandler(void) {
    HAL_DMA_IRQHandler(&handle_GPDMA1_Channel1);  // triggers GetDataCallback
}

void GPDMA1_Channel0_IRQHandler(void) {
    HAL_DMA_IRQHandler(&handle_GPDMA1_Channel0);  // triggers DataReadyCallback
}

void JPEG_IRQHandler(void) {
    HAL_JPEG_IRQHandler(&hjpeg);  // triggers EncodeCpltCallback /
ErrorCallback
}
```

8.2 IRQ Priority Configuration

```
// From MX_GPDMA1_Init() and HAL_JPEG_MspInit()
HAL_NVIC_SetPriority(GPDMA1_Channel0_IRQn, 0, 0);
HAL_NVIC_EnableIRQ(GPDMA1_Channel0_IRQn);
HAL_NVIC_SetPriority(GPDMA1_Channel1_IRQn, 0, 0);
HAL_NVIC_EnableIRQ(GPDMA1_Channel1_IRQn);
HAL_NVIC_SetPriority(JPEG_IRQn, 0, 0);
HAL_NVIC_EnableIRQ(JPEG_IRQn);
```

> ⚠ **Note:** All three IRQs are at priority 0 (highest) in this example. In a production system with an RTOS or other peripherals, adjust priorities to avoid starving the JPEG DMA — a stalled DMA stalls the hardware FIFO which stalls the entire encoding pipeline.

8.3 The IRQ/CPU Contract

The architecture strictly separates responsibilities: IRQ callbacks are lightweight signal-only functions that change flags and pause/reconfigure DMA. All heavy work (RGB conversion, memcpy, resume decisions) runs in CPU context inside the background loop. This keeps interrupt latency minimal and avoids race conditions from doing heap allocation or long operations inside ISRs.

Callback (IRQ context)	Only does	Signals CPU via
HAL_JPEG_GetDataCallback	Reconfigure DMA window, or Pause + set EMPTY	Jpeg_IN_BufferTab.State, Input_Is_Paused
HAL_JPEG_DataReadyCallback	Set FULL flag, Pause output DMA, reconfig output buffer	Jpeg_OUT_BufferTab.State, Output_Is_Paused
HAL_JPEG_EncodeCpltCallback	Set end flag	Jpeg_HWEncodingEnd

9. Porting to Another STM32

9.1 What Must Change

Item	STM32H5 (this example)	STM32H7	STM32F7 (F76/77)
DMA peripheral	GPDMA1	MDMA or DMA2 streams	DMA2 streams
DMA IN request	GPDMA1_REQUEST_JPEG_RX_REQ	DMA_REQUEST_JPEG_IN	DMA_CHANNEL_9 (DMA2 St5)
DMA OUT request	GPDMA1_REQUEST_JPEG_TX_REQ	DMA_REQUEST_JPEG_OUT	DMA_CHANNEL_9 (DMA2 St4)
IRQ names	GPDMA1_Channel0/1_IRQn	DMA2_StreamX_IRQn	DMA2_StreamX_IRQn
JPEG IRQ	JPEG_IRQn	JPEG_IRQn (same)	JPEG_IRQn (same)
DMA2D YCbCr→RGB	Yes (Chrom-ART2)	Yes (Chrom-ART)	No — software only
TrustZone	Yes (H5E5/F5)	No	No

9.2 What Stays the Same

- The entire encode_dma.c state machine logic — callbacks, handlers, buffer management
- All HAL_JPEG_* API calls: HAL_JPEG_Init, HAL_JPEG_ConfigEncoding, HAL_JPEG_Encode_DMA, HAL_JPEG_Pause, HAL_JPEG_Resume, HAL_JPEG_ConfigInputBuffer, HAL_JPEG_ConfigOutputBuffer
- jpeg_utils_conf.h pixel format defines
- The jpeg_utils library (jpeg_utils.c / jpeg_utils.h) — copy from ST's JPEG utility package
- main loop structure, timeout logic, LED status reporting

9.3 Porting Checklist

1. Enable HAL_JPEG_MODULE_ENABLED and HAL_DMA_MODULE_ENABLED in stm32xxxx_hal_conf.h
2. Update HAL_JPEG_MspInit() — change GPDMA1 to DMA2/MDMA, update request names, stream/channel numbers
3. Update IRQ handler names in stm32xxxx_it.c to match the DMA streams on your MCU
4. Recalculate PLL settings in SystemClock_Config() for your board's HSE crystal frequency
5. Verify Flash latency setting in the reference manual for your target SYSCLK
6. Copy jpeg_utils.c, jpeg_utils.h, and jpeg_utils_conf.h from ST's JPEG utility package (not in standard HAL)

7. Replace Image_ARGB8888[] / Image_RGB888[] / Image_RGB565[] with your own image arrays
8. Replace BSP_LED_On() calls with your board's GPIO or remove them
9. Verify total SRAM usage fits in your target MCU — IN buffers + OUT buffers + jpgBuffer
10. Use STM32CubeMX peripheral table to confirm your specific part number includes the JPEG peripheral

10. Common Issues and Debugging

Symptom	Likely Cause	Fix
Error_Handler() called immediately	Image dimensions not multiples of 8/16	Pad image to next multiple of 16 (both axes for 4:2:0)
Green LED never lights — 5s timeout	DMA not configured — JPEG hardware stalls	Check GPDMA1_REQUEST_JPEG_RX/TX_REQ request IDs and channel linkage
Corrupted JPEG output	Wrong BYTES_PER_PIXEL or JPEG_RGB_FORMAT mismatch	Verify jpeg_utils_conf.h matches actual image pixel format
Encoding freezes mid-image	RTOS task starving IRQ — DMA bursts not acknowledged	Raise JPEG and DMA IRQ priorities; ensure SysTick priority is lower
Output JPEG file unreadable	jpgBuffer overflow — final pointer past allocated size	Increase jpgBuffer or reduce image quality; add bounds check in OutputHandler
MCU_BlockIndex never reaches MCU_TotalNb	MAX_INPUT_LINES or image size mismatch with subsampling mode	Check MAX_INPUT_LINES = 16 for 4:2:0, 8 for others
HAL_JPEG_Encode_DMA returns error	JPEG peripheral clock not enabled in MspInit	Verify __HAL_RCC_JPEG_CLK_ENABLE() is called in HAL_JPEG_MspInit

11. Quick Reference

11.1 Key Formulas

Value	Formula	Example (320×240 ARGB8888 4:2:0)
DataBufferSize (RGB bytes / chunk)	ImageWidth × MAX_INPUT_LINES × BPP	320 × 16 × 4 = 20 480 bytes
MCU_TotalNb	(W ÷ MCU_W) × (H ÷ MCU_H)	(320÷16) × (240÷16) = 300 MCUs
Chunks per image	ImageHeight ÷ MAX_INPUT_LINES	240 ÷ 16 = 15 chunks
DMA bursts per chunk	DataBufferSize ÷ 32	20 480 ÷ 32 = 640 bursts
GetDataCallback calls (full image)	Bursts × Chunks	640 × 15 = 9 600 calls
CHUNK_SIZE_IN (buffer alloc)	MAX_INPUT_WIDTH × MAX_INPUT_LINES × BPP	800 × 16 × 4 = 51 200 bytes

11.2 Configuration Decision Tree

```
Source pixel format?
  ├─ 4 bytes/pixel (ARGB, RGBA)  → JPEG_ARGB8888, BYTES_PER_PIXEL=4
  ├─ 3 bytes/pixel (RGB, BGR)    → JPEG_RGB888,   BYTES_PER_PIXEL=3
  └─ 2 bytes/pixel (RGB565)      → JPEG_RGB565,   BYTES_PER_PIXEL=2

Quality vs size priority?
  ├─ Best quality (medical, text) → 4:4:4, MAX_INPUT_LINES=8,  quality=90+
  ├─ Balanced (general photo)     → 4:2:0, MAX_INPUT_LINES=16, quality=75
  └─ Smallest file (thumbnail)    → 4:2:0, MAX_INPUT_LINES=16, quality=40-60

Image dimensions must satisfy:
  Width  % 8  == 0  (always)
  Height % 8  == 0  (always)
  Width  % 16 == 0  (4:2:2 and 4:2:0)
  Height % 16 == 0  (4:2:0 only)
```